

Instructor: Date:

Learner: Regional Branch:

Program: Center:

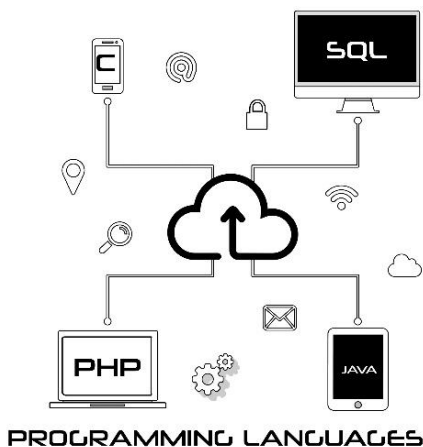
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Understanding Declarative Languages in Software Programming



Learning Outcome:

Understand key information about the concepts, principles, and methodologies in simple spoken and written situations. Talk about programming techniques, use cases, kinds, and sequences in simple spoken and written situations.

Can Do Statements:

- I can talk about the declarative languages in software programming.
- I can list some declarative languages.
- I can give examples of declarative languages.

Ask your partner:

- In what cases do you think the contextualization of the program is relevant?
- What online applications do you like using? Why?



Let's Explore!

A student is taking a tutoring session about declarative languages.



Instructions: Read the text below.

Mike: So, Paul, what do you usually do in your free time?

Paul: I'm very fond of programming software systems; so, I often look for up-to-date information about programming methods.

Mike: Cool, do you know anything about those programming paradigms that define the programming languages?

Paul: Of course! At the moment, I'm reading a magazine article about declarative programming; I sometimes read magazines that specializes on programming languages.

Mike: Declarative programming? Which paradigm is that one?

Paul: Well, declarative programming is an umbrella term that groups some paradigms different from the imperative programming paradigm; what those alternative paradigms have in common is that they do not describe how the program is going to work, but what the program is going to accomplish.

Mike: What other paradigms does the term declarative programming cover?

Paul: Some of the programming paradigms covered by the concept of declarative languages are: constraint programming which executes the program as a set of constraints, or conditions that the problem domain solves by giving a certain value to each variable. There are also the domain-specific languages which are very useful languages without being computationally universal. What those languages have in common is that they don't list commands to perform in a given order, but they give each variable its own meaning; for that reason, we can understand them as isolated declarations.

Mike: Cool! I heard that, to some degree, declarative languages simplify the process of programming because they focus on the goal rather than on the process.

Paul: That's true, those languages are context independent; this is, any instructional steps are irrelevant. Thus, we can use the program in different contexts.

Mike: In my work, I sometimes use declarative languages, such as Absys, Alpha, Datalog, Erlang, SGNAL, and SQL.

Paul: Those are well-known declarative languages; I use them all the time.





Let's Practice!

How do we define declarative programming?

Instructions: Classify the sentences by writing the number of the sentence in the correct category.

1. The source code describes what the program aims to do.
2. The source code declares how to do something in a program.
3. With this type of programming, we don't need to list the steps or make the sequence of commands of a program.
4. These languages use logical statements that are proved during the execution.
5. This type of response provides a set of instructions to do something.

These are common features of Declarative Languages.	These aren't common features of Declarative Languages.



Let's Practice!

Here we have some examples of declarative languages.

Instructions: Select the correct answer in order to complete the sentence.

1. Functional and logical languages _____ examples of declarative programming.
 - a. is
 - b. are
 - c. aren't
2. For instance, if a computer program _____ to solve the question: How _____ I get to the closest bookstore? ...
 - a. aim/is
 - b. aims/do
 - c. aims/does
3. The declarative response _____ you the precise address of the place you need to go.
 - a. give
 - b. gives
 - c. giving
4. When a programmer writes an HTML page, the HTML tags only _____ the elements that need to appear on the web page.
 - a. declare
 - b. declares
 - c. Declaring
5. HTML tags _____ specify the control flow of the page.
 - a. never
 - b. rarely
 - c. frequently



Extension Activity

Creating a Manual about Declarative Languages in Software Programming.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Read more information about declarative languages in software programming.
- Identify key ideas and take notes.
- Create a short manual to explain how declarative languages work.
- Give examples
- Use the simple present and the frequency adverbs.
- Share your ideas with your classmates.

Self-Assessment

Yes

No

- I read more information about declarative languages in software programming.
- I identify key ideas and take notes.
- I create a short manual.
- I give examples.
- I use the grammar covered in this lesson.
- I share my work with my classmates.

☐	☐
☐	☐
☐	☐
☐	☐
☐	☐
☐	☐



Target Words

- programming paradigm** (noun): a specific style of programming
e.g. This term generally makes reference to a specific programming paradigm or style, in which the program's source code describes what the program aims to do instead of declaring how to do it.
- aim** (verb): to expect a specific result
e.g. This term generally makes reference to a specific programming paradigm or style, in which the program's source code describes what the program aims to do instead of declaring how to do it.
- figure out** (phrasal verb): to understand something
e.g. Well, this job is left to the compiler, it has to figure out how the program is going to accomplish its main objective.

- sequence** (noun): a list of related items
e.g. What you are saying is that in declarative programming, we don't need to list the steps or make the sequence of commands of a program.
- Markup language** (noun): a type of programming language in which the program commands are syntactically different from the text of a file
e.g. Markup languages such as HTML or XML are often declarative.



Learning Points

Adverbs of Frequency

You can use the adverbs of frequency with the simple present to describe how often something happens.

The **adverbs of frequency** have different degrees of regularity; never, for example, means 0%; and always makes reference to a 100% of frequency.

When you use the adverbs of frequency you need to consider the position of the verbs:

Subject + **adverb of frequency** + verb + complement.

Examples:

We **generally** send the information in the morning.

They **normally** use different programming languages.

But when you use the verb **to be**, this is the structure you use:

Subject+ **verb to be** + **adverb of frequency** + complement.

Examples:

We **are often** at the office at that time.

She **is always** talking about programming.

Frequency	Adverbs of frequency
100%	Always
90%	Usually
70%	Normally
50%	Often
40%	Sometimes
30%	Occasionally
10%	Seldom
0%	Never

Examples

- This term generally makes reference to a specific programming paradigm or style, in which the program's source code describes what the program aims to do instead of declaring how to do it, as it normally happens in imperative programming.
- That is why declarative languages such as functional and logical programming languages often use a predicate logic.



Learning Points

Simple Present Tense

We use **the simple present tense** to talk about habits and routines, general facts, and permanent states.

We use this tense to talk about:

Habits and routines:

- *He usually **uses** HTML to write web pages.*
- *He **doesn't** usually **use** HTML to write web pages.*
- ***Does** he usually **use** HTML to write web pages?*

General facts and permanent states:

- *Declarative languages **work** with logical statements.*
- *Declarative languages **don't work** with logical statements.*
- ***Do** declarative languages **work** with logical statements?*

The following are the structures you use to make sentences in the **simple present tense**:

Affirmative Form

Subject + **verb in simple present** + complement.

Example:

- *A declarative language only defines the expected result.*

Note that in affirmative statements, you need to add **-s**, **-es** or **-ies** to the verbs when you use the third person in singular.

Negative Form

Subject + **do not (don't)/does not (doesn't)** + **verb in base form** + complement.

Example:

- *Declarative languages **don't** use sequences of commands.*

In negative statements, you need to use the auxiliary **doesn't** with the third person in singular: she, he, and it. For the other pronouns, you need to use the auxiliary **don't**.

Interrogative Form

Wh-word (optional) + **do/does** + **verb in base form** + complement.

Example:

- *How **does** declarative programming **work**?*

Note that the use of the auxiliary **does** in questions with the third person in singular: she, he, and *it*. For the other pronouns, you need to use the auxiliary **do**.





Learning Points

Examples

- *How do we define declarative programming?*
- *What you are saying is that in declarative programming, we don't need to list the steps or make the sequence of commands of a program.*
- *When a programmer writes an HTML page, the HTML tags only declare the elements that need to appear on the web page or application.*

Types of Computer Programmers

There are different types of programmer careers in the computer industry.

Computer Programmers

Computer programmers are in charge of writing software products for several fields of work. They have to define the objectives of each program and code the instructions in the order that the program is able to execute them.

Game Programmers

These type of programmers write computer games; they use low-level and close-to-the-machine programming languages so as to allow games to run faster on PCs or consoles.

Web Programmers

They are also known as web developers and they build web pages and applications. They generally use web development languages such as HTML, Java, C++, among others.

Web developers also need to work hard implementing security in their programs due to hacking risks that may exist on the internet.

